

Curso _____
Sistemas Operativos

**UNIVERSIDAD
DE ANTIOQUIA**

Clase 18 - Semaforos (Parte 1)

Agenda

- Resumen clase anterior
- Contextualización
- Semáforos
- Semáforos binarios (locks)
- Semáforos como variables de condición

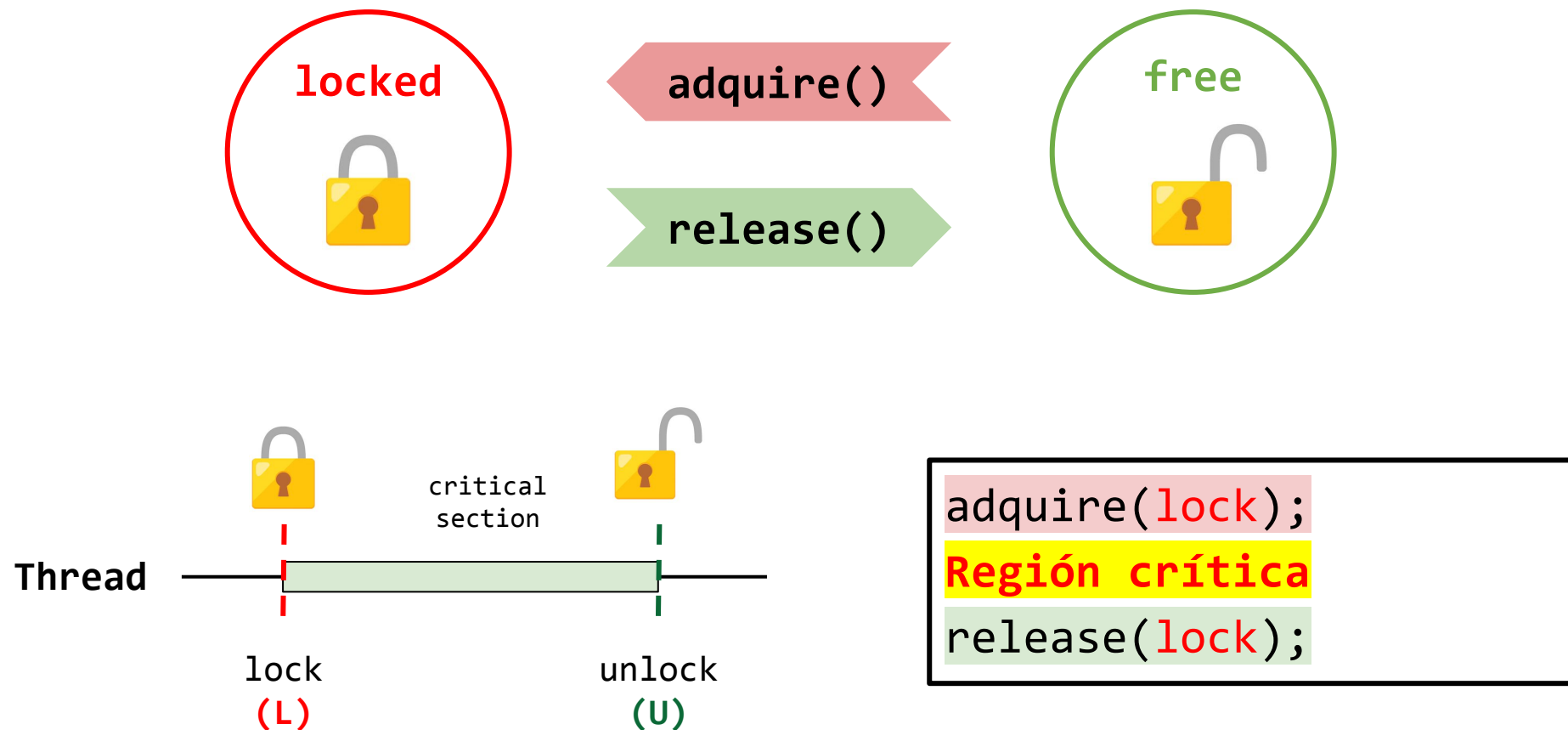
Agenda

- **Resumen clase anterior**
- Contextualización
- Semáforos
- Semáforos binarios (locks)
- Semáforos como variables de condición

Resumen clase anterior

Locks

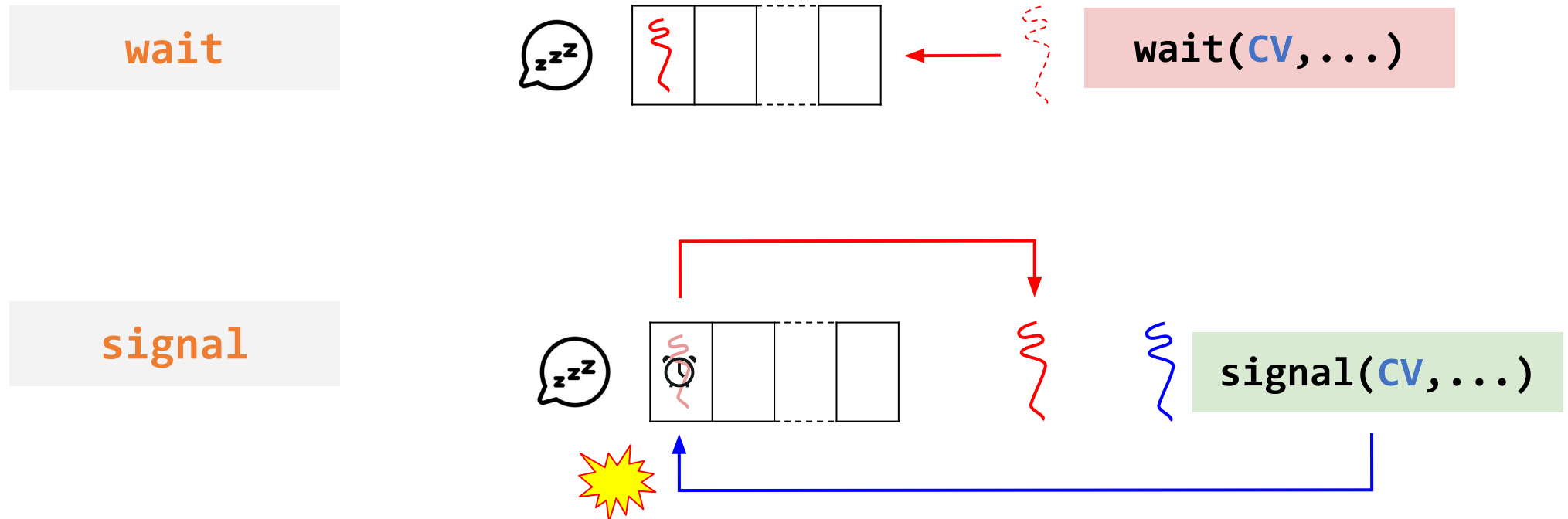
- Un lock es un objeto de memoria que proporciona exclusión mutua.



Resumen clase anterior

Variables de condición (CV)

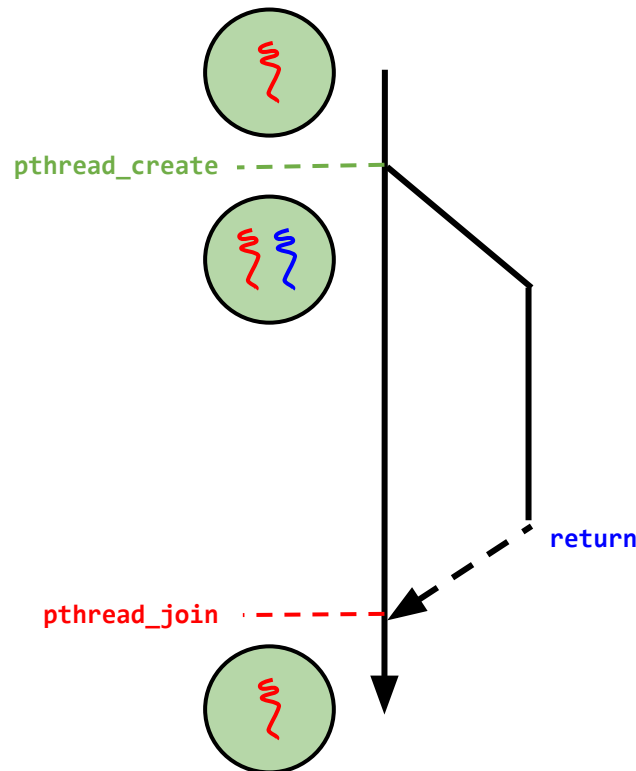
- Las variables de condición son un tipo de datos abstracto.
- Cola de hilos que se encuentran dormidos.



Resumen clase anterior

¿Cómo implementar el orden de ejecución deseado?

- ¿Cómo implementar el join? (Para lograr el orden de ejecución deseado).



```
void *child(void *arg) {  
    printf("child\n");  
    // XXX how to indicate we are done?  
    return NULL;  
}  
  
int main(int argc, char *argv[]) {  
    printf("parent: begin\n");  
    pthread_t c;  
    Pthread_create(&c, NULL, child, NULL); //create child  
    // XXX how to wait for child?  
    printf("parent: end\n");  
    return 0;  
}
```

parent: begin
child
parent: end

Salida deseada

Código



Resumen clase anterior

¿Cómo implementar el orden de ejecución deseado?

Implementación del join usando variables de condición

```
int done = 0;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
void *child(void *arg) {
    printf("child\n");
    thr_exit();
    return NULL;
}
int main(int argc, char *argv[]) {
    printf("parent: begin\n");
    pthread_t p;
    pthread_create(&p, NULL, child, NULL);
    thr_join();
    printf("parent: end\n");
    return 0;
}
```

```
void thr_exit() {
    pthread_mutex_lock(&m);
    done = 1;
    pthread_cond_signal(&c);
    pthread_mutex_unlock(&m);
}
```

```
void thr_join() {
    pthread_mutex_lock(&m);
    while (done == 0)
        pthread_cond_wait(&c, &m);
    pthread_mutex_unlock(&m);
}
```



Agenda

- Resumen clase anterior
- **Contextualización**
- Semáforos
- Semáforos binarios (locks)
- Semáforos como variables de condición

Contextualización



- Hemos visto hasta el momento como mediante el uso de **locks** y **variables de condición** se puede dar solución a problemas que implican concurrencia.
- En esta sección se va a estudiar una nueva primitiva de sincronización conocida como **semáforo** propuesta por Dijkstra en 1968 (**Goto Statements Considered Harmful** - [link](#))

Pregunta clave

¿Cómo usar semáforos?

- ¿Cómo se puede usar semáforos en lugar de **locks** y **variables de condición**?
- ¿Cual es la definición de un **semáforo**?
- ¿Que es un **semáforo binario**?
- ¿Es sencillo construir un **semáforo** usando **locks** y **variables de condición**?
- ¿Es posible construir **locks** y **variables de condición** a partir de **semáforos**?



Agenda

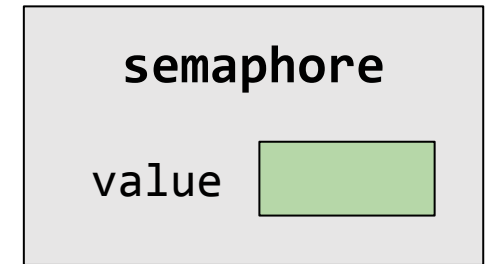
- Resumen clase anterior
- Contextualización
- **Semáforos**
- Semáforos binarios (locks)
- Semáforos como variables de condición

Semáforos

Aspectos claves

- Primitiva de sincronización inventada por Dijkstra en 1968.
- **Objeto con un valor entero** no negativo (asociado al estado).

semaphore S



Operaciones

Un **semáforo**, solo puede ser manipulado por a través de dos operaciones.

sem_wait(): También conocida como **wait()**, **P()** o **down()**.



```
wait(S) {
    while (S <= 0)
        ; // busy wait
    S--;
}
```

sem_post(): También conocida como **signal()**, **V()** o **up()**.

```
signal(S) {
    S++;
}
```

Semáforos

Operaciones con semáforos

1. Inicialización

Antes de interactuar con cualquier semáforo es necesario declararlo e inicializarlo.

```
sem_t sem;  
int sem_init(&sem, is_shared, init_value);
```

Ejemplo:

```
#include <semaphore.h>  
sem_t s;  
sem_init(&s, 0, 1); // initialize s to the value 1
```

1. Declaración del semáforo `s`
2. Inicialización del semáforo:
 - El valor inicial es asignado a 1 (tercer argumento)
 - El segundo argumento indica que el semáforo es compartido entre los hilos de un mismo proceso.



Semáforos

Operaciones con semáforos

2. wait o test

Si el valor del semáforo es:

- **Mayor o igual a uno (≥ 1)**, la función retorna inmediatamente.
- **En caso contrario**, el hilo llamador se suspende (wait) a la espera de una señal de post.

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

Un semáforo con un valor negativo indica el número de hilos en espera.

```
int sem_wait (sem_t *s) {  
    decrement the value of semaphore s by one  
    wait if value of semaphore s is negative  
}
```

Semáforos

Operaciones con semáforos

3. Post o signal o increment

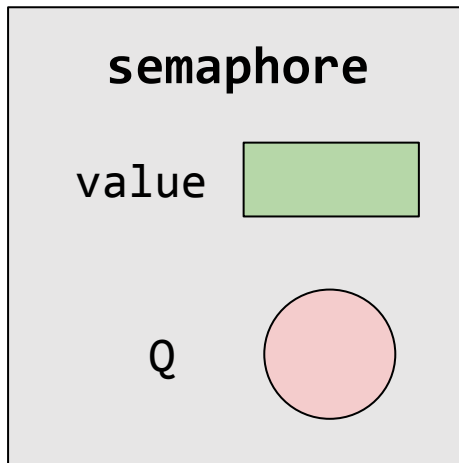
- **Incrementa** el valor del semáforo
- Si hay hilos a la espera en el semáforo, **despierta alguno de ellos**.

```
signal(S) {  
    S++;  
}
```

```
int sem_post(sem_t *s) {  
    increment the value of semaphore s by one  
    if there are one or more threads waiting, wake one  
}
```

Semáforos

Operaciones con semáforos - Resumen y pseudocódigo



```
typedef struct {  
    int value;  
    struct process *Q;  
} sem_t;
```

init

```
sem_init(sem_t *s, int value) {  
    s->value = initval  
}
```

wait

```
void sem_wait(sem_t *s) { // Must be executed atomically  
    s->value--;  
    if (s->value < 0) {  
        add this process to s->Q;  
        block();  
    }  
}
```

post

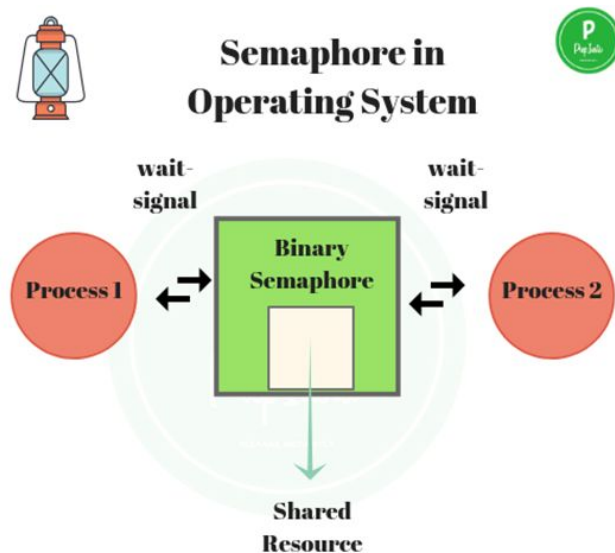
```
void sem_post(sem_t *s) { // Must be executed atomically  
    s->value++;  
    if (s->value <= 0) {  
        remove a process P from s->Q;  
        wakeup(P);  
    }  
}
```

Semáforos

Tipos de semáforos

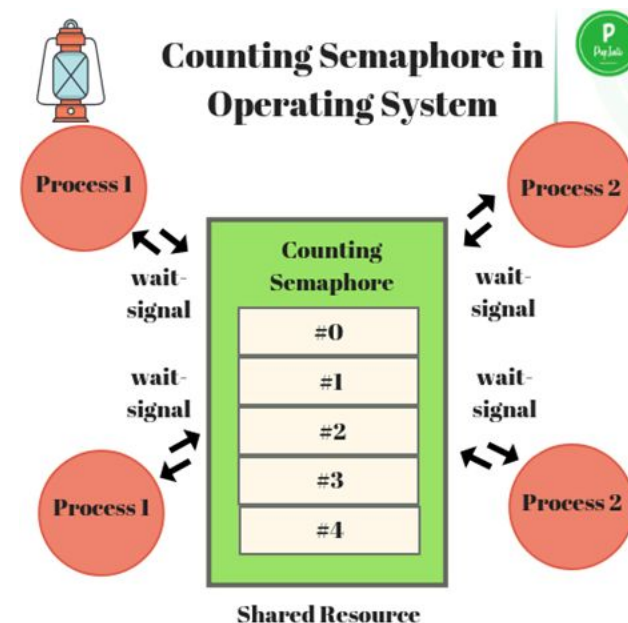
Semáforos binarios ($\approx$ mutex)

- Semáforo cuyo valor es inicializado a 1.
- Si hay hilos a la espera en el semáforo, **despierta alguno de ellos**.
- Solo se permite la entrada de un hilo a la vez.



Semáforo general (counting semaphore)

- Semáforo cuyo valor es inicializado a **N**
- Representa un recurso con muchas unidades disponibles.
- Permite la entrada de hilos siempre que haya más unidades disponibles.



Agenda

- Resumen clase anterior
- Contextualización
- Semáforos
- **Semáforos binarios (locks)**
- Semáforos como variables de condición

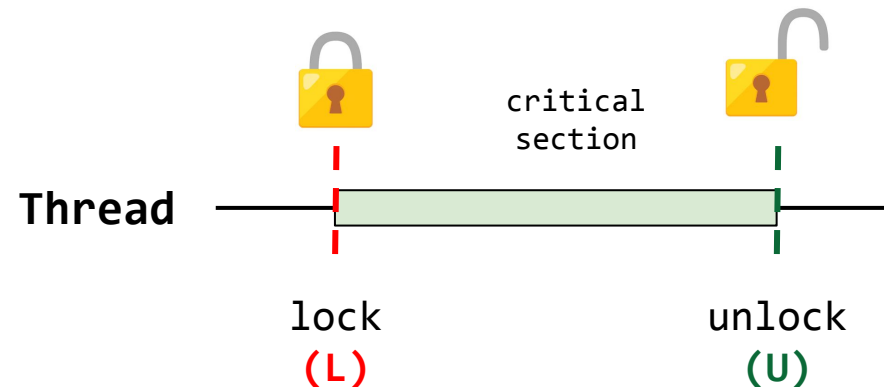
Semáforos binarios (locks)

- Un **semáforo** puede ser usado como **lock**.
- La sección crítica va entre las operaciones **wait()** y **post()**

```
sem_t m;  
sem_init (&m, 0 , X); // initialize semaphore to X; what should X be?  
sem_wait (&m);  
//critical section here  
sem_post (&m);
```

Para el caso: ¿Cuál debe ser el valor de X?

El valor inicial debe ser: ____



Semáforos binarios (locks)

Ejemplo: Un hilo, un semáforo

- Escenario con dos hilos (**Thread 0** y **Thread 1**)
- De los dos hilos, solo uno de estos usa el semáforo.

Thread 0



Thread 1



```
sem_t m;
sem_init (&m, 0 , 1);
sem_wait (&m);
//critical section here
sem_post (&m);
```

Value of Semaphore	Thread 0	Thread 1
1		
1	call sem_wait()	
0	sem_wait() returns	
0	(crit sect)	
0	call sem_post()	
1	sem_post() returns	

Thread Trace: Single Thread Using A Semaphore

Semáforos binarios (locks)

Ejemplo: Dos hilos, un semáforo

- Escenario con dos hilos (**Thread 0** y **Thread 1**)
- De los dos hilos:
 - **Thread 0**: Ha llamado a `sem_wait()` pero aún no ha llamado a `sem_post()` - **holds the lock**.
 - **Thread 1**: Intenta ingresar en la **región crítica** llamando a `sem_post()`.

Thread 0



Thread 1

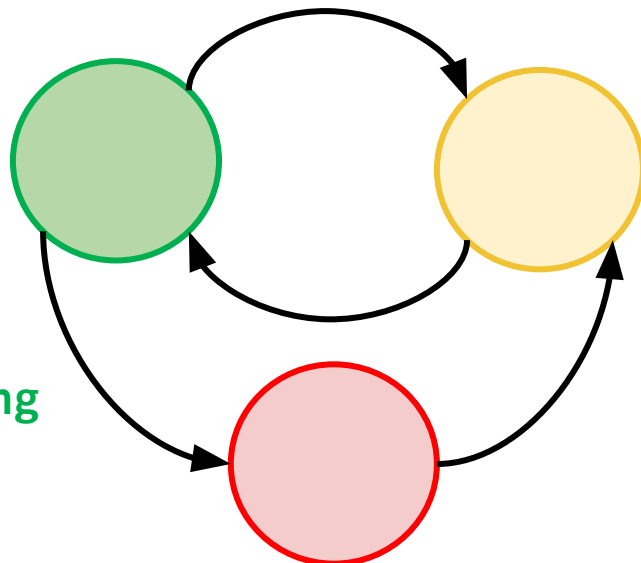


```
sem_t m;  
sem_init (&m, 0 , 1);  
sem_wait (&m);  
//critical section here  
sem_post (&m);
```

Semáforos binarios (locks)

Ejemplo: Dos hilos, un semáforo

```
sem_t m;
sem_init (&m, 0 , 1);
sem_wait (&m);
//critical section here
sem_post (&m);
```



Value	Thread 0	State	Thread 1	State
1		Running		Ready
1	call sem_wait()	Running		Ready
0	sem_wait() retruns	Running		Ready
0	(crit set: begin)	Running		Ready
0	Interrupt; Switch → T1	Ready		Running
0		Ready	call sem_wait()	Running
-1		Ready	decrement sem	Running
-1		Ready	(sem < 0) → sleep	sleeping
-1		Running	Switch → T0	sleeping
-1	(crit sect: end)	Running		sleeping
-1	call sem_post()	Running		sleeping
0	increment sem	Running		sleeping
0	wake (T1)	Running		Ready
0	sem_post() returns	Running		Ready
0	Interrupt; Switch → T1	Ready		Running
0		Ready	sem_wait() retruns	Running
0		Ready	(crit sect)	Running
0		Ready	call sem_post()	Running
1		Ready	sem_post() returns	Running

Semáforos binarios (locks)

Ejemplo: Dos hilos, un semáforo

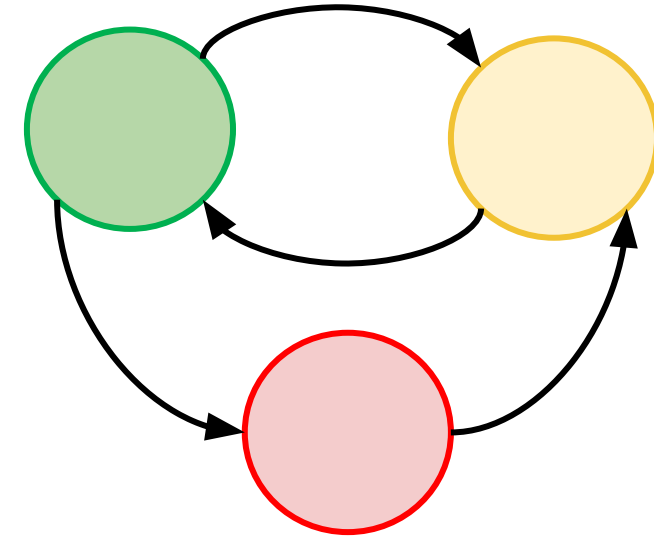
~ ~

```

a      sem_init (&m, 0 , 1);

1      sem_wait (&m);
x      //critical section here
y      sem_post (&m);
    
```

Running
Ready
Sleep



value(sem)	1	1	0	0	0	0	-1	-1	-1	-1	-1	0	0	0	0	0	0	0	1
Thread 0	a	1->w	w->	->x						x->	y->p		p->						
Thread 1	a					1->w	w->									w->	->x	y->p	p->
sleep	---	---	---	---	---	---	T1	T1	T1	T1	T1	---	---	---					

Convención:

- Llamado a una función o región crítica: linea->f / ->f
- Retorno de una función o región crítica: f->
- wait: w
- post: p



Semáforos binarios (locks)

Código de implementación del lock empleando semáforos

- Un semáforo binario tiene un valor booleano (no entero).
- **Operaciones:**
 - **bsem_wait():** Espera hasta que el valor sea 1. Luego se establece en 0.
 - **bsem_signal():** Establece el valor en 1 y despierta uno de los procesos que está esperando

```
sem_init(sem_t *s, int initval) {  
    s->value = initval  
}
```

```
sem_wait(sem_t *s) {  
    while (s->value <= 0)  
        put_self_to_sleep();  
    s->value--;  
}
```

```
sem_post(sem_t *s) {  
    s->value++;  
    wake_one_waiting_thread();  
}
```

```
typedef struct __lock_t {  
    // whatever data structs you need goes here  
} lock_t;
```

```
void init(lock_t *lock) {  
    // init code goes here  
}
```

```
void acquire(lock_t *lock) {  
    // lock acquire code goes here  
}
```

```
void release(lock_t *lock) {  
    // lock release code goes here  
}
```



Semáforos binarios (locks)

Código de implementación del lock empleando semáforos

```
sem_init(sem_t *s, int initval) {
    s->value = initval
}

sem_wait(sem_t *s) {
    while (s->value <= 0)
        put_self_to_sleep();
    s->value--;
}

sem_post(sem_t *s) {
    s->value++;
    wake_one_waiting_thread();
}
```

```
typedef struct __lock_t {
    // whatever data structs you need goes here
    sem_t value;
} lock_t;

void init(lock_t *lock) {
    // init code goes here
    sem_init(&lock->value, 0, 1);
}

void acquire(lock_t *lock) {
    // lock acquire code goes here
    sem_wait(&lock->value);
}

void release(lock_t *lock) {
    // lock release code goes here
    sem_post(&lock->value);
}
```

Semáforos binarios (locks)

Código de implementación del lock empleando semáforos - Código de test

```
sem_t lock;
void *worker(void *arg) {
    int i;
    sem_wait(&lock); // What goes here?
    for (i = 0; i < 1e6; i++)
        counter++;
    sem_post(&lock); // What goes here?
    return NULL;
}

int main(int argc, char *argv[]) {
    int num = atoi(argv[1]);
    pthread_t pid[PMAX];
    sem_init(&lock, 0, 1 /* What goes here? */);
    for (int i = 0; i < num; i++)
        Pthread_create(&pid[i], 0, worker, 0);
    for (int i = 0; i < num; i++)
        Pthread_join(pid[i], NULL);
    printf("counter: %d\n", counter);
    return 0;
}
```

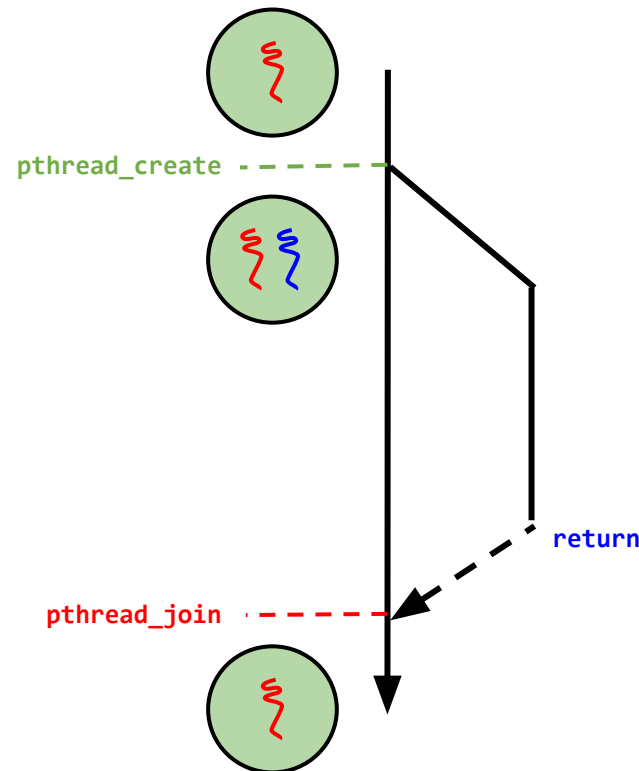


Agenda

- Resumen clase anterior
- Contextualización
- Semáforos
- Semáforos binarios (locks)
- **Semáforos como variables de condición**

Semáforos como variables de condición

- Los semáforos también son útiles para **ordenar eventos** en un programa concurrente lo que los hace aptos para ser usados como **primitiva para controlar el orden de ejecución**.



```
sem_t s;
void* child(void*arg) {
    printf("child\n");
    sem_post(&s); // signal here: child is done
    return NULL;
}

int main(intargc, char*argv[]) {
    sem_init(&s, 0, X); // what should X be?
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s); // wait here for child
    printf("parent: end\n");
    return 0;
}
```

```
parent: begin
child
parent: end
```

¿Cómo usar el semáforo para alcanzar este efecto?

- Simplemente el padre llama a `sem_wait()` y el hijo llama a `sem_post()`.

¿Cuál debería ser el valor de x?

- El valor del semáforo debería ser llevado a ____



Semáforos como variables de condición

Análisis: El padre espera al hijo

Caso 1: Padre llama `sem_wait()` **antes** que hijo llama `sem_post()`.

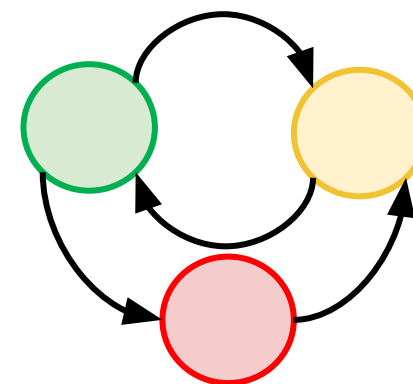
```
sem_t s;
void* child(void*arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(intargc, char*argv[]) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

Val	Parent	State	Child	State
0	create (Child)	Run	(Child exists; can run)	Ready
0	Interrupt→Child	Ready	child runs	Run
0		Ready	call sem_post()	Run
1		Ready	inc sem	Run
1		Ready	wake (nobody)	Run
1		Ready	sem_post() returns	Run
1	parent runs	Run	Interrupt→Parent	Ready
1	call sem_wait()	Run		Ready
0	decrement sem	Run		Ready
0	(sem ≥ 0) → awake	Run		Ready
0	sem_wait() returns	Run		Ready

Thread Trace: Parent Waiting For Child (Case 2)

Running
Ready
Sleep



s->value

?



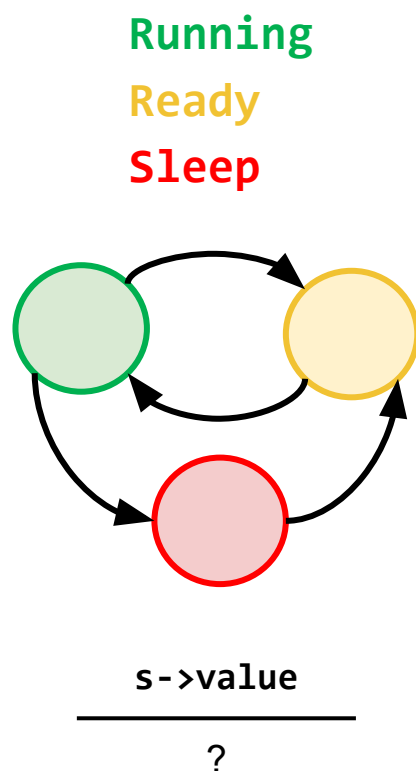
Semáforos como variables de condición

Análisis: El padre espera al hijo

Caso 1: Padre llama `sem_wait()` **antes** que hijo llama `sem_post()`.

```
sem_t s;
void* child(void*arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(intargc, char*argv[]) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```



Val(sem)	Parent (P)	Child (C)	sleep
0	create(C)		---
0			---
0		->p()	---
1		sem++	---
1		wake(nobody)	---
1		p()->	---
1			---
1	->w()		---
0	sem--		---
0	(sem>=0)->awake		---
0	w()->		---

Semáforos como variables de condición

Análisis: El padre espera al hijo

Caso 2: Padre llama `sem_wait()`
después que hijo llama `sem_post()`.

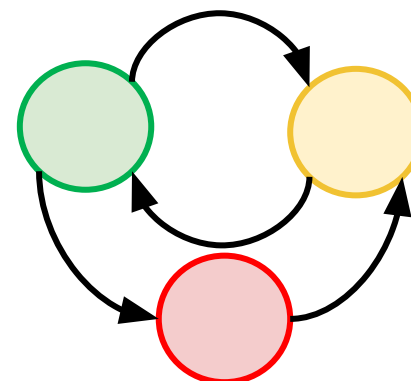
```
sem_t s;
void* child(void*arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(intargc, char*argv[]) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t c;
    pthread_create(&c, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```

Val	Parent	State	Child	State
0	create (Child)	Run	(Child exists, can run)	Ready
0	call sem_wait()	Run		Ready
-1	decr sem	Run		Ready
-1	(sem<0) → sleep	Sleep		Ready
-1	Switch → Child	Sleep	child runs	Run
-1		Sleep	call sem_post()	Run
0		Sleep	inc sem	Run
0		Ready	wake (Parent)	Run
0		Ready	sem_post() returns	Run
0		Ready	Interrupt → Parent	Ready
0	sem_wait() returns	Run		Ready

Thread Trace: Parent Waiting For Child (Case 1)

Running
Ready
Sleep



s->value

?



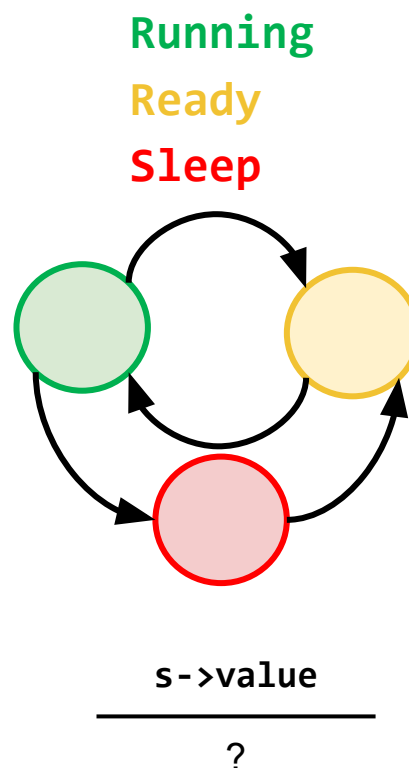
Semáforos como variables de condición

Análisis: El padre espera al hijo

Caso 2: Padre llama `sem_wait()` **después** que hijo llama `sem_post()`.

```
sem_t s;
void* child(void*arg) {
    printf("child\n");
    sem_post(&s);
    return NULL;
}

int main(intargc, char*argv[]) {
    sem_init(&s, 0, 0);
    printf("parent: begin\n");
    pthread_t tc;
    pthread_create(&tc, NULL, child, NULL);
    sem_wait(&s);
    printf("parent: end\n");
    return 0;
}
```



Val(sem)	Parent (P)	Child (C)	sleep
0	create(C)		---
0	->wait()		---
-1	sem--		---
-1	(sem<0)->sleep		P
-1		child runs	P
-1		->post()	P
0		sem++	P
0		wake(P)	---
0		post()->	---
0			---
0	wait()->		---

Semáforos como variables de condición

Comparación:

La implementación del **join** (con el fin de hacer que el padre espere por el hijo) es más simple con **semáforos** que con **variables de condición (CV)**.

```
int done = 0;
mutex_t m = MUTEX_INIT;
cond_t c = COND_INIT;
void *child(void *arg) {
    printf("child\n");
    mutex_lock(&m);
    done = 1;
    cond_signal(&c);
    mutex_unlock(&m);
}
int main(int argc, char *argv[]) {
    pthread_t c;
    printf("parent: begin\n");
    pthread_create(c, NULL, child, NULL);
    mutex_lock(&m);
    while(done == 0)
        cond_wait(&c, &m);
    mutex_unlock(&m);
    printf("parent: end\n");
}
```

Implementación con variables de Condición (CV)

```
sem_t s;
void *child(void *arg) {
    printf("child\n");
    sem_post(&s); // increment
}
int main(int argc, char *argv[]) {
    sem_init(&s, 0, 0);
    pthread_t c;
    printf("parent: begin\n");
    pthread_create(c, NULL, child, NULL);
    sem_wait(&s); // decrement, wait for counter == 0
    printf("parent: end\n");
}
```

Implementación con semáforos



Referencias

- Diapositivas y videos oficiales del curso
- <https://adit.io/posts/2013-05-15-Locks,-Actors,-And-STM-In-Pictures.html>
- https://link.springer.com/chapter/10.1007/978-1-4842-4398-5_5
- <https://preshing.com/20150316/semaphores-are-surprisingly-versatile/>
- <https://jvns.ca/teach-tech-with-cartoons/>
- <https://code-cartoons.com/>
- <https://medium.com/basecs>
- <https://xkcd.com/>
- <https://www.qwantz.com/>
- <https://pbfcomics.com/>
- <http://turnoff.us/>
- <http://turnoff.us/geek/arduino-project/>
- <https://prepinsta.com/operating-systems/critical-section-problem/>
- <https://github.com/capedrazab/os-20191/wiki>
- <https://github.com/jjpulidos/Operating-Systems-UNAL-20191>
- <https://github.com/capedrazab/os-20201/wiki>



Referencias

- <http://csl.snu.ac.kr/courses/4190.307/2020-1/>
- <https://brendangregg.com/linuxperf.html>
- <http://pages.cs.wisc.edu/~harter/537/slides.html>
- <http://www.embeddedlinux.org.cn/rtconforembsys/5107final/LiB0053.html>
- [http://odl.sysworks.biz/disk\\$axpdocsep012/network/dcev30/develop/appdev/Appde179.htm](http://odl.sysworks.biz/disk$axpdocsep012/network/dcev30/develop/appdev/Appde179.htm)
- <https://pages.mtu.edu/~shene/NSF-3/e-Book/MONITOR/monitor-types.html>
- <https://ocw.unican.es/course/view.php?id=240§ion=1>
- <https://omscs.gatech.edu/cs-6200-introduction-operating-systems>
- <https://ruslanspivak.com/lbaws-part1/>
- <https://prepinsta.com/operating-systems/threads/>
- <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/implementing-threads/>

Referencias

- <https://github.com/capedrazab/os-20211>
- <https://github.com/pkgamma/cs241-coursebook-clone-fa19/wiki/Synchronization>
- <https://github.com/tiemma/udacity-operating-systems>
- <https://github.com/monorinam/GaTech-CS-6210-Advanced-Operating-Systems>
- <https://github.com/stevenxchung/Introduction-to-Operating-Systems>
- <https://github.com/stevenxchung/Introduction-to-Operating-Systems/tree/master/Lesson%2014%20-%20Synchronization%20Constructs>
- <https://github.com/khanhnamle1994/operating-systems>
- <https://github.com/gauti1311/SystemMonitor>
- https://github.com/bresilla/robond_project_2
- <https://github.com/Maxxquoi/Introduction-to-Operating-Systems>
- <https://github.com/sergioarmgpl/operating-systems-usac-course>
- <https://github.com/MrinmoiHossain/Online-Courses-Learning>
- <https://github.com/PKUFlyingPig/MIT6.S081-2020fall>
- <https://github.com/mishal23/os-lab>



Referencias

- <https://github.com/mishal23/os-simulator>
- <https://github.com/zxc479773533/HUST-OS-Course-Design>
- <https://github.com/mauri870/baking-pi>
- <https://github.com/hyojeonglee/osfall2019>
- <https://github.com/JoyeBright/OSLab>
- <https://github.com/Matrixpecker/VE482-public>
- <https://github.com/MrinmoiHossain/Robot-Operating-System-Udemy>
- <https://github.com/Aniruddha-Tapas/Operating-Systems-Notes>
- <https://github.com/Aniruddha-Tapas/Operating-Systems-Notes/blob/master/3-Threads-and-Concurrency.md>
- <http://www.embeddedlinux.org.cn/rtconforembsys/5107final/LiB0053.html>
- <https://levelup.gitconnected.com/producer-consumer-problem-using-condition-variable-in-c-6c4d96efcbbc>
- <http://jakascorner.com/blog/2016/01/condition-variable.html>
- <https://pages.mtu.edu/~shene/NSF-3/e-Book/MONITOR/monitor-types.html>
- <https://github.com/CodeExMachina/ConditionVariable>



Referencias

- <http://csl.snu.ac.kr/courses/4190.307/2020-1/>
- <https://brendangregg.com/linuxperf.html>
- <http://pages.cs.wisc.edu/~harter/537/slides.html>
- <http://csl.snu.ac.kr/courses/4190.307/2020-1/>
- http://ferestrepoca.github.io/paradigmas-de-programacion/paralela/paralela_teorias/index.html
- <https://prepinsta.com/operating-systems/semaphore/>
- <https://www.geeksforgeeks.org/semaphores-in-process-synchronization/>
- <https://preshing.com/20150316/semaphores-are-surprisingly-versatile/>
- <https://prepinsta.com/operating-systems/mutex-vs-semaphore/>
- <https://computationstructures.org/lectures/synchronization/synchronization.html>
- <https://www.slideshare.net/mohammedarif89/semaphores>
- <http://pages.cs.wisc.edu/~harter/537/slides.html>
- <http://www.embeddedlinux.org.cn/rtconforembsys/5107final/LiB0053.html>



Referencias

- <https://pages.mtu.edu/~shene/NSF-3/e-Book/MONITOR/monitor-types.html>
- <https://ocw.unican.es/course/view.php?id=240§ion=1>
- <https://omscs.gatech.edu/cs-6200-introduction-operating-systems>
- <https://ruslanspivak.com/lbaws-part1/>
- <https://prepinsta.com/operating-systems/threads/>
- <http://www.it.uu.se/education/course/homepage/os/vt18/module-4/implementing-threads/>

UNIVERSIDAD DE ANTIOQUIA

Curso de sistemas Operativos
Clase 18 - Semáforos (Parte 1)